

Gestión de Notas en Ensamblador 8086

Integrantes: Alejandro Obando
Josue Chango
Jardel Maza

Nrc: 202650

21 de junio de 2026

1. Objetivo del Proyecto

Desarrollar un programa en ensamblador 8086 que permita gestionar las notas de un grupo de estudiantes. El programa debe solicitar la cantidad de estudiantes a las que se les asignara notas, leer las notas individuales (válidas de 00 a 20), calcular el promedio total, la nota más alta y la nota más baja, y finalmente mostrar un resumen con los resultados.

2. Descripción Técnica

2.1. Planteamiento del problema

Se requiere un sistema que permita al usuario ingresar la cantidad de estudiantes (entre 1 y 9) y posteriormente sus notas correspondientes, valide que cada nota esté en el rango de 0 a 20, y finalmente calcule y muestre el promedio, la nota más alta y la nota más baja del grupo de estudiantes. Todo esto debe implementarse en ensamblador 8086 con el emulador emu8086.

2.2. Algoritmo utilizado

Cabe recalcar que este algoritmo es construido a partir de algoritmos vistos y resueltos en clase.

El algoritmo sigue los siguientes pasos:

1. Solicitar al usuario la cantidad de estudiantes.
2. Para cada estudiante:
 - a) Solicitar la nota como dos dígitos individuales (decena y unidad).
 - b) Validar que la nota esté entre 00 y 20.
 - c) Si es inválida, mostrar mensaje de error y repetir la solicitud.
3. Convertir cada par de dígitos a su valor numérico multiplicando la decena por 10 y sumando la unidad.
4. Calcular la suma total de las notas.
5. Determinar la nota máxima y mínima mediante comparaciones sucesivas.
6. Calcular el promedio mediante restas sucesivas (división entera).
7. Mostrar el resumen: promedio, nota más alta y nota más baja.

2.3. Instrucciones clave del 8086 utilizadas

- MOV: Transferencia de datos entre registros y memoria.
- CMP: Comparación de dos operandos para la toma de decisiones.
- JMP, JE, JG, JGE, JL, JNE: Saltos condicionales e incondicionales para el control de flujo.

- INT 21h: Llamadas al sistema operativo para entrada/salida.
- ADD, SUB, DEC, INC: Operaciones aritméticas.
- LEA: Carga de direcciones efectivas para el manejo de cadenas.

3. Implementación

3.1. Herramientas utilizadas

- emu8086 — Emulador y ensamblador para microprocesadores 8086.
- Editor de texto para la escritura del código fuente y respectivo informe.

3.2. Código fuente

A continuación se presenta el código completo del programa con comentarios explicativos:

```
1  org 100h
2
3  ; ***** SOLICITAR CANTIDAD DE ESTUDIANTES *****
4
5  lea dx, msg_cantidad
6  mov ah, 9
7  int 21h
8
9  mov ah, 01h
10 int 21h
11 sub al, 48      ; Convertir ASCII a numero
12 mov cantidad, al ; Guardar cantidad de estudiantes
13
14 mov contador, 0 ; Inicializar contador de notas
15
16 ; ***** LECTURA DE NOTAS (pedir1 a pedir10) *****
17 ; Cada bloque: pide decena+unidad, valida rango 00-20,
18 ; si es invalida muestra error y repite y si es valida avanza.
19
20 pedir1:
21     mov al, contador
22     cmp al, cantidad
23     je fin_lectura
24
25     lea dx, msg_nota
26     mov ah, 9
27     int 21h
28
29     mov ah, 01h
30     int 21h
31     sub al, 48
32     mov decena1, al
33
34     mov ah, 01h
35     int 21h
36     sub al, 48
37     mov unidad1, al
```

```

38
39     mov al, decena1
40     cmp al, 2
41     jg invalida1
42     cmp al, 2
43     jne valida1
44     mov al, unidad1
45     cmp al, 0
46     jne invalida1
47     jmp valida1
48
49 invalida1:
50     lea dx, msg_error
51     mov ah, 9
52     int 21h
53     jmp pedir1
54
55 valida1:
56     inc contador
57     jmp pedir2
58
59 pedir2:
60     mov al, contador
61     cmp al, cantidad
62     je fin_lectura
63
64     lea dx, msg_nota
65     mov ah, 9
66     int 21h
67
68     mov ah, 01h
69     int 21h
70     sub al, 48
71     mov decena2, al
72
73     mov ah, 01h
74     int 21h
75     sub al, 48
76     mov unidad2, al
77
78     mov al, decena2
79     cmp al, 2
80     jg invalida2
81     cmp al, 2
82     jne valida2
83     mov al, unidad2
84     cmp al, 0
85     jne invalida2
86     jmp valida2
87
88 invalida2:
89     lea dx, msg_error
90     mov ah, 9
91     int 21h
92     jmp pedir2
93
94 valida2:
95     inc contador

```

```

96     jmp pedir3
97
98 pedir3:
99     mov al, contador
100    cmp al, cantidad
101    je fin_lectura
102
103    lea dx, msg_nota
104    mov ah, 9
105    int 21h
106
107    mov ah, 01h
108    int 21h
109    sub al, 48
110    mov decena3, al
111
112    mov ah, 01h
113    int 21h
114    sub al, 48
115    mov unidad3, al
116
117    mov al, decena3
118    cmp al, 2
119    jg invalida3
120    cmp al, 2
121    jne valida3
122    mov al, unidad3
123    cmp al, 0
124    jne invalida3
125    jmp valida3
126
127 invalida3:
128    lea dx, msg_error
129    mov ah, 9
130    int 21h
131    jmp pedir3
132
133 valida3:
134    inc contador
135    jmp pedir4
136
137 pedir4:
138    mov al, contador
139    cmp al, cantidad
140    je fin_lectura
141
142    lea dx, msg_nota
143    mov ah, 9
144    int 21h
145
146    mov ah, 01h
147    int 21h
148    sub al, 48
149    mov decena4, al
150
151    mov ah, 01h
152    int 21h
153    sub al, 48

```

```

154     mov unidad4, al
155
156     mov al, decena4
157     cmp al, 2
158     jg invalida4
159     cmp al, 2
160     jne valida4
161     mov al, unidad4
162     cmp al, 0
163     jne invalida4
164     jmp valida4
165
166 invalida4:
167     lea dx, msg_error
168     mov ah, 9
169     int 21h
170     jmp pedir4
171
172 valida4:
173     inc contador
174     jmp pedir5
175
176 pedir5:
177     mov al, contador
178     cmp al, cantidad
179     je fin_lectura
180
181     lea dx, msg_nota
182     mov ah, 9
183     int 21h
184
185     mov ah, 01h
186     int 21h
187     sub al, 48
188     mov decena5, al
189
190     mov ah, 01h
191     int 21h
192     sub al, 48
193     mov unidad5, al
194
195     mov al, decena5
196     cmp al, 2
197     jg invalida5
198     cmp al, 2
199     jne valida5
200     mov al, unidad5
201     cmp al, 0
202     jne invalida5
203     jmp valida5
204
205 invalida5:
206     lea dx, msg_error
207     mov ah, 9
208     int 21h
209     jmp pedir5
210
211 valida5:

```

```

212     inc contador
213     jmp pedir6
214
215 pedir6:
216     mov al, contador
217     cmp al, cantidad
218     je fin_lectura
219
220     lea dx, msg_nota
221     mov ah, 9
222     int 21h
223
224     mov ah, 01h
225     int 21h
226     sub al, 48
227     mov decena6, al
228
229     mov ah, 01h
230     int 21h
231     sub al, 48
232     mov unidad6, al
233
234     mov al, decena6
235     cmp al, 2
236     jg invalida6
237     cmp al, 2
238     jne valida6
239     mov al, unidad6
240     cmp al, 0
241     jne invalida6
242     jmp valida6
243
244 invalida6:
245     lea dx, msg_error
246     mov ah, 9
247     int 21h
248     jmp pedir6
249
250 valida6:
251     inc contador
252     jmp pedir7
253
254 pedir7:
255     mov al, contador
256     cmp al, cantidad
257     je fin_lectura
258
259     lea dx, msg_nota
260     mov ah, 9
261     int 21h
262
263     mov ah, 01h
264     int 21h
265     sub al, 48
266     mov decena7, al
267
268     mov ah, 01h
269     int 21h

```

```

270     sub al, 48
271     mov unidad7, al
272
273     mov al, decena7
274     cmp al, 2
275     jg invalida7
276     cmp al, 2
277     jne valida7
278     mov al, unidad7
279     cmp al, 0
280     jne invalida7
281     jmp valida7
282
283 invalida7:
284     lea dx, msg_error
285     mov ah, 9
286     int 21h
287     jmp pedir7
288
289 valida7:
290     inc contador
291     jmp pedir8
292
293 pedir8:
294     mov al, contador
295     cmp al, cantidad
296     je fin_lectura
297
298     lea dx, msg_nota
299     mov ah, 9
300     int 21h
301
302     mov ah, 01h
303     int 21h
304     sub al, 48
305     mov decena8, al
306
307     mov ah, 01h
308     int 21h
309     sub al, 48
310     mov unidad8, al
311
312     mov al, decena8
313     cmp al, 2
314     jg invalida8
315     cmp al, 2
316     jne valida8
317     mov al, unidad8
318     cmp al, 0
319     jne invalida8
320     jmp valida8
321
322 invalida8:
323     lea dx, msg_error
324     mov ah, 9
325     int 21h
326     jmp pedir8
327

```



```

328 valida8:
329     inc contador
330     jmp pedir9
331
332 pedir9:
333     mov al, contador
334     cmp al, cantidad
335     je fin_lectura
336
337     lea dx, msg_nota
338     mov ah, 9
339     int 21h
340
341     mov ah, 01h
342     int 21h
343     sub al, 48
344     mov decena9, al
345
346     mov ah, 01h
347     int 21h
348     sub al, 48
349     mov unidad9, al
350
351     mov al, decena9
352     cmp al, 2
353     jg invalida9
354     cmp al, 2
355     jne valida9
356     mov al, unidad9
357     cmp al, 0
358     jne invalida9
359     jmp valida9
360
361 invalida9:
362     lea dx, msg_error
363     mov ah, 9
364     int 21h
365     jmp pedir9
366
367 valida9:
368     inc contador
369     jmp pedir10
370
371 pedir10:
372     mov al, contador
373     cmp al, cantidad
374     je fin_lectura
375
376     lea dx, msg_nota
377     mov ah, 9
378     int 21h
379
380     mov ah, 01h
381     int 21h
382     sub al, 48
383     mov decena10, al
384
385     mov ah, 01h

```

```

386     int 21h
387     sub al, 48
388     mov unidad10, al
389
390     mov al, decena10
391     cmp al, 2
392     jg invalida10
393     cmp al, 2
394     jne valida10
395     mov al, unidad10
396     cmp al, 0
397     jne invalida10
398     jmp valida10
399
400 invalida10:
401     lea dx, msg_error
402     mov ah, 9
403     int 21h
404     jmp pedir10
405
406 valida10:
407     inc contador
408     jmp fin_lectura
409
410 ; ***** CONVERTIR DIGITOS A VALORES NUMERICOS *****
411 ; Multiplica decena x 10 (sumando 10 veces) y suma la unidad
412
413 fin_lectura:
414     mov al, 0
415     mov bl, decena1
416     mov cl, 10
417 sumar_d1:
418     cmp cl, 0
419     je listo_d1
420     add al, bl
421     dec cl
422     jmp sumar_d1
423 listo_d1:
424     add al, unidad1
425     mov valor1, al
426
427     mov al, 0
428     mov bl, decena2
429     mov cl, 10
430 sumar_d2:
431     cmp cl, 0
432     je listo_d2
433     add al, bl
434     dec cl
435     jmp sumar_d2
436 listo_d2:
437     add al, unidad2
438     mov valor2, al
439
440     mov al, 0
441     mov bl, decena3
442     mov cl, 10
443 sumar_d3:

```

```

444     cmp cl, 0
445     je listo_d3
446     add al, bl
447     dec cl
448     jmp sumar_d3
449 listo_d3:
450     add al, unidad3
451     mov valor3, al
452
453     mov al, 0
454     mov bl, decena4
455     mov cl, 10
456 sumar_d4:
457     cmp cl, 0
458     je listo_d4
459     add al, bl
460     dec cl
461     jmp sumar_d4
462 listo_d4:
463     add al, unidad4
464     mov valor4, al
465
466     mov al, 0
467     mov bl, decena5
468     mov cl, 10
469 sumar_d5:
470     cmp cl, 0
471     je listo_d5
472     add al, bl
473     dec cl
474     jmp sumar_d5
475 listo_d5:
476     add al, unidad5
477     mov valor5, al
478
479     mov al, 0
480     mov bl, decena6
481     mov cl, 10
482 sumar_d6:
483     cmp cl, 0
484     je listo_d6
485     add al, bl
486     dec cl
487     jmp sumar_d6
488 listo_d6:
489     add al, unidad6
490     mov valor6, al
491
492     mov al, 0
493     mov bl, decena7
494     mov cl, 10
495 sumar_d7:
496     cmp cl, 0
497     je listo_d7
498     add al, bl
499     dec cl
500     jmp sumar_d7
501 listo_d7:

```

```

502     add al, unidad7
503     mov valor7, al
504
505     mov al, 0
506     mov bl, decena8
507     mov cl, 10
508 sumar_d8:
509     cmp cl, 0
510     je listo_d8
511     add al, bl
512     dec cl
513     jmp sumar_d8
514 listo_d8:
515     add al, unidad8
516     mov valor8, al
517
518     mov al, 0
519     mov bl, decena9
520     mov cl, 10
521 sumar_d9:
522     cmp cl, 0
523     je listo_d9
524     add al, bl
525     dec cl
526     jmp sumar_d9
527 listo_d9:
528     add al, unidad9
529     mov valor9, al
530
531     mov al, 0
532     mov bl, decena10
533     mov cl, 10
534 sumar_d10:
535     cmp cl, 0
536     je listo_d10
537     add al, bl
538     dec cl
539     jmp sumar_d10
540 listo_d10:
541     add al, unidad10
542     mov valor10, al
543
544 ; ***** CALCULAR SUMA, MAXIMA Y MINIMA *****
545 ; Recorre las notas acumulando suma y actualizando max/min
546
547     mov suma, 0
548     mov maxima, 0
549     mov minima, 99
550     mov contador, 0
551
552     mov al, contador
553     cmp al, cantidad
554     jge fin_calculo
555     mov al, valor1
556     add suma, al
557     cmp al, maxima
558     jle chk_min_1
559     mov maxima, al

```

```

560
561 chk_min_1:
562     cmp al, minima
563     jge sig_1
564     mov minima, al
565
566 sig_1:
567     inc contador
568     mov al, contador
569     cmp al, cantidad
570     jge fin_calculo
571     mov al, valor2
572     add suma, al
573     cmp al, maxima
574     jle chk_min_2
575     mov maxima, al
576
577 chk_min_2:
578     cmp al, minima
579     jge sig_2
580     mov minima, al
581
582 sig_2:
583     inc contador
584     mov al, contador
585     cmp al, cantidad
586     jge fin_calculo
587     mov al, valor3
588     add suma, al
589     cmp al, maxima
590     jle chk_min_3
591     mov maxima, al
592
593 chk_min_3:
594     cmp al, minima
595     jge sig_3
596     mov minima, al
597
598 sig_3:
599     inc contador
600     mov al, contador
601     cmp al, cantidad
602     jge fin_calculo
603     mov al, valor4
604     add suma, al
605     cmp al, maxima
606     jle chk_min_4
607     mov maxima, al
608
609 chk_min_4:
610     cmp al, minima
611     jge sig_4
612     mov minima, al
613
614 sig_4:
615     inc contador
616     mov al, contador
617     cmp al, cantidad

```

```

618     jge fin_calculo
619     mov al, valor5
620     add suma, al
621     cmp al, maxima
622     jle chk_min_5
623     mov maxima, al
624
625 chk_min_5:
626     cmp al, minima
627     jge sig_5
628     mov minima, al
629
630 sig_5:
631     inc contador
632     mov al, contador
633     cmp al, cantidad
634     jge fin_calculo
635     mov al, valor6
636     add suma, al
637     cmp al, maxima
638     jle chk_min_6
639     mov maxima, al
640
641 chk_min_6:
642     cmp al, minima
643     jge sig_6
644     mov minima, al
645
646 sig_6:
647     inc contador
648     mov al, contador
649     cmp al, cantidad
650     jge fin_calculo
651     mov al, valor7
652     add suma, al
653     cmp al, maxima
654     jle chk_min_7
655     mov maxima, al
656
657 chk_min_7:
658     cmp al, minima
659     jge sig_7
660     mov minima, al
661
662 sig_7:
663     inc contador
664     mov al, contador
665     cmp al, cantidad
666     jge fin_calculo
667     mov al, valor8
668     add suma, al
669     cmp al, maxima
670     jle chk_min_8
671     mov maxima, al
672
673 chk_min_8:
674     cmp al, minima
675     jge sig_8

```

```

676     mov minima, al
677
678 sig_8:
679     inc contador
680     mov al, contador
681     cmp al, cantidad
682     jge fin_calculo
683     mov al, valor9
684     add suma, al
685     cmp al, maxima
686     jle chk_min_9
687     mov maxima, al
688
689 chk_min_9:
690     cmp al, minima
691     jge sig_9
692     mov minima, al
693
694 sig_9:
695     inc contador
696     mov al, contador
697     cmp al, cantidad
698     jge fin_calculo
699     mov al, valor10
700     add suma, al
701     cmp al, maxima
702     jle chk_min_10
703     mov maxima, al
704
705 chk_min_10:
706     cmp al, minima
707     jge sig_10
708     mov minima, al
709
710 sig_10:
711     inc contador
712
713 ; ***** CALCULAR PROMEDIO *****
714 ; Resta 'cantidad' de 'suma' hasta que ya no se pueda;
715 ; la cantidad de restas es el promedio.
716
717 fin_calculo:
718     mov promedio, 0
719     mov al, suma
720
721 restar_promedio:
722     mov bl, cantidad
723     cmp al, bl
724     jb fin_promedio
725     sub al, bl
726     inc promedio
727     jmp restar_promedio
728
729 ; ***** MOSTRAR RESULTADOS *****
730 ; Convierte cada valor a 2 digitos restando 10 sucesivamente
731 ; y muestra: Promedio, Nota mas alta, Nota mas baja.
732
733 fin_promedio:

```

```

734     lea dx, msg_resumen
735     mov ah, 9
736     int 21h
737     lea dx, msg_promedio
738     mov ah, 9
739     int 21h
740     mov al, promedio
741     mov bl, 0
742
743 mostrar_dec_prom:
744     cmp al, 10
745     jl listo_dec_prom
746     sub al, 10
747     inc bl
748     jmp mostrar_dec_prom
749
750 listo_dec_prom:
751     mov temp_unidad, al
752     mov dl, bl
753     add dl, 48
754     mov ah, 2
755     int 21h
756     mov dl, temp_unidad
757     add dl, 48
758     mov ah, 2
759     int 21h
760     lea dx, msg_maxima
761     mov ah, 9
762     int 21h
763     mov al, maxima
764     mov bl, 0
765
766 mostrar_dec_max:
767     cmp al, 10
768     jl listo_dec_max
769     sub al, 10
770     inc bl
771     jmp mostrar_dec_max
772
773 listo_dec_max:
774     mov temp_unidad, al
775     mov dl, bl
776     add dl, 48
777     mov ah, 2
778     int 21h
779     mov dl, temp_unidad
780     add dl, 48
781     mov ah, 2
782     int 21h
783     lea dx, msg_minima
784     mov ah, 9
785     int 21h
786     mov al, minima
787     mov bl, 0
788
789 mostrar_dec_min:
790     cmp al, 10
791     jl listo_dec_min

```



```

792     sub al, 10
793     inc bl
794     jmp mostrar_dec_min
795
796 listo_dec_min:
797     mov temp_unidad, al
798     mov dl, bl
799     add dl, 48
800     mov ah, 2
801     int 21h
802     mov dl, temp_unidad
803     add dl, 48
804     mov ah, 2
805     int 21h
806
807 ret
808
809 ; ***** VARIABLES Y MENSAJES *****
810
811 msg_cantidad db 0Dh,0Ah,"Cuantos estudiantes? (1-9): $"
812 msg_nota     db 0Dh,0Ah,"Ingrese nota como 2 digitos, ej 15 = 1 y 5: $"
813 msg_error    db 0Dh,0Ah,"Nota invalida (debe ser 00 a 20). Intente de nuevo: $"
814 msg_resumen  db 0Dh,0Ah,0Ah,"-----RESUMEN----- $"
815 msg_promedio db 0Dh,0Ah,"Promedio: $"
816 msg_maxima   db 0Dh,0Ah,"Nota mas alta: $"
817 msg_minima   db 0Dh,0Ah,"Nota mas baja: $"
818
819 cantidad db 0
820 contador db 0
821
822 decena1 db 0
823 unidad1 db 0
824 decena2 db 0
825 unidad2 db 0
826 decena3 db 0
827 unidad3 db 0
828 decena4 db 0
829 unidad4 db 0
830 decena5 db 0
831 unidad5 db 0
832 decena6 db 0
833 unidad6 db 0
834 decena7 db 0
835 unidad7 db 0
836 decena8 db 0
837 unidad8 db 0
838 decena9 db 0
839 unidad9 db 0
840 decena10 db 0
841 unidad10 db 0
842
843 valor1 db 0
844 valor2 db 0
845 valor3 db 0
846 valor4 db 0
847 valor5 db 0
848 valor6 db 0

```

```

849 valor7 db 0
850 valor8 db 0
851 valor9 db 0
852 valor10 db 0
853
854 suma db 0
855 promedio db 0
856 maxima db 0
857 minima db 99
858 temp_unidad db 0

```

4. Pruebas y Resultados

4.1. Casos de prueba

Caso	Cant. estudiantes	Notas	Promedio esperado	Max/Min esperados
1	3	15, 18, 20	17	20 / 15
2	4	10, 12, 14, 16	13	16 / 10
3	5	20, 20, 20, 20, 20	20	20 / 20
4	2	05, 00	2	05 / 00
5	3	07, 11, 14	10	14 / 07

Cuadro 1: Casos de prueba del programa.

4.2. Ejecución del programa

A continuación se presentan las capturas de pantalla de cada caso de prueba ejecutado en emu8086:

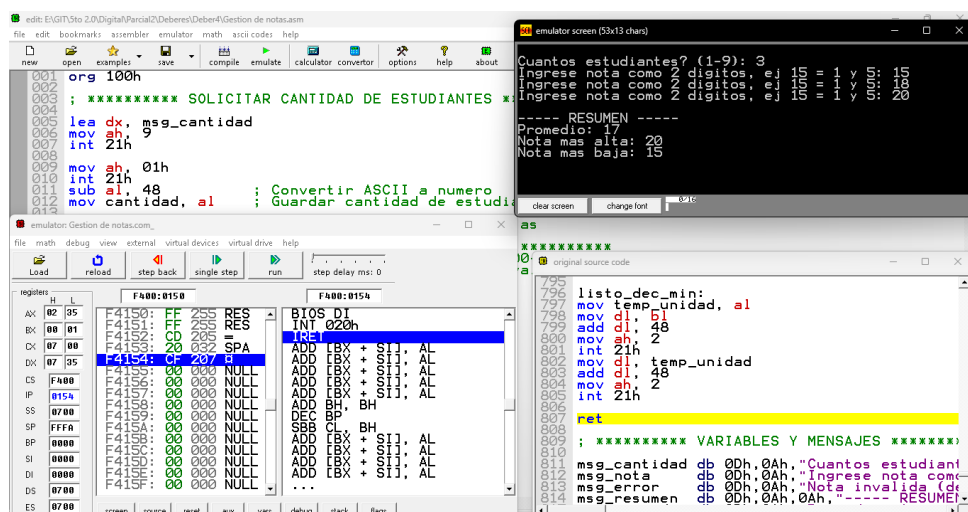


Figura 1: Caso 1: 3 estudiantes con notas 15, 18, 20. Promedio esperado: 17, Max: 20, Min: 15.

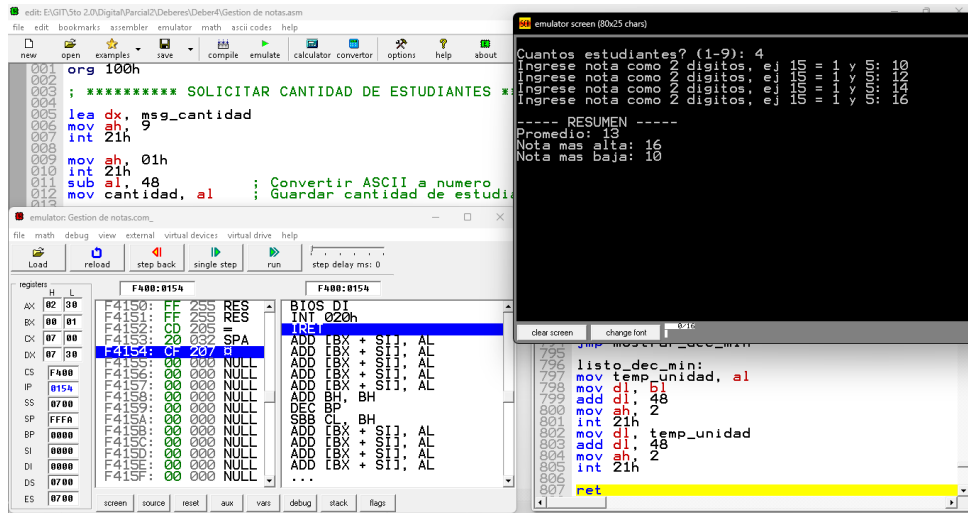


Figura 2: Caso 2: 4 estudiantes con notas 10, 12, 14, 16. Promedio esperado: 13, Max: 16, Min: 10.

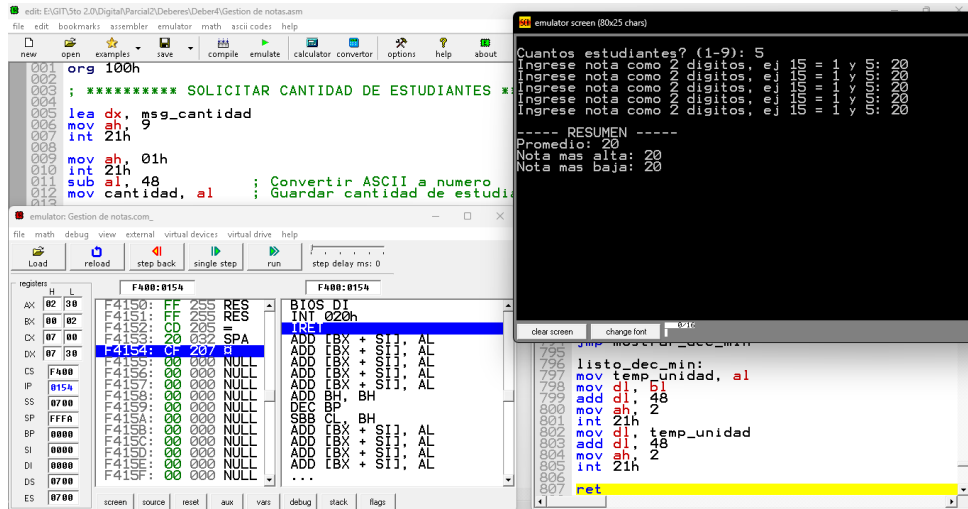


Figura 3: Caso 3: 5 estudiantes con notas 20, 20, 20, 20, 20. Promedio esperado: 20, Max: 20, Min: 20.

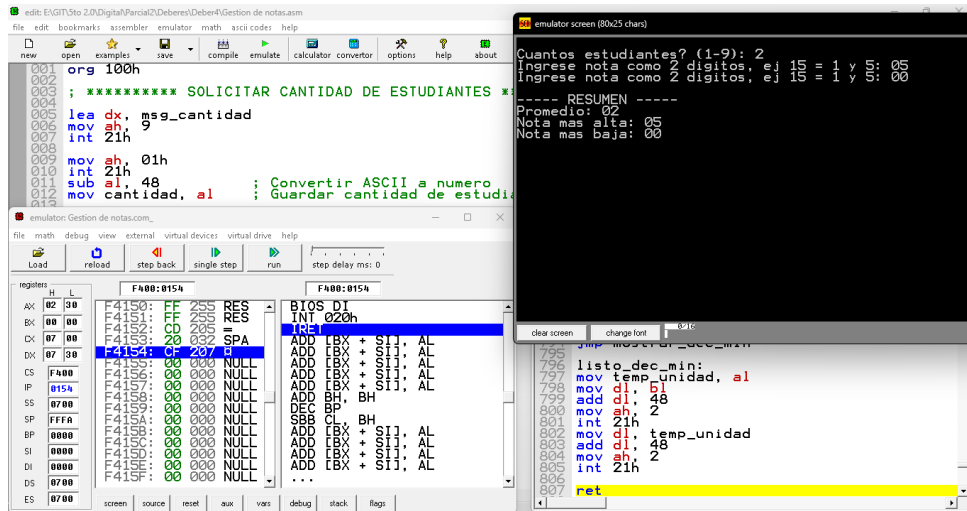


Figura 4: Caso 4: 2 estudiantes con notas 05, 00. Promedio esperado: 2, Max: 05, Min: 00.

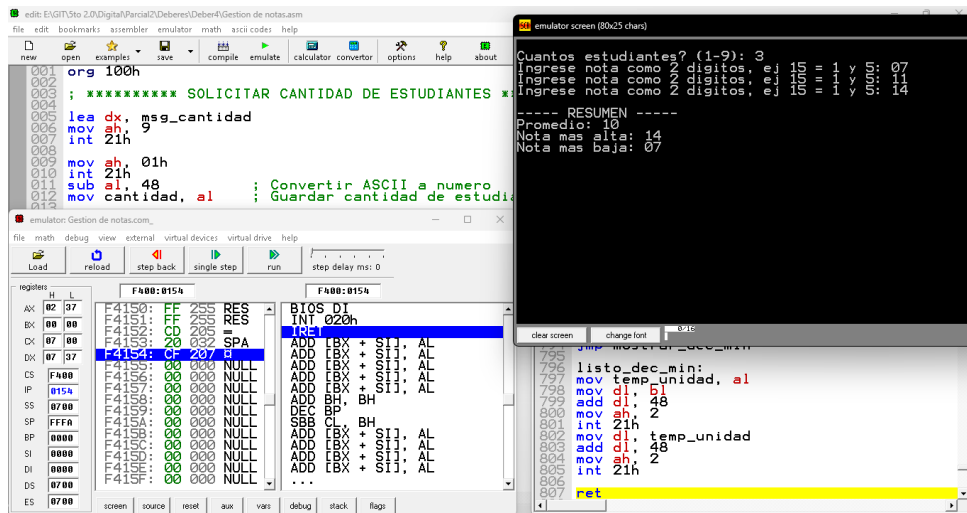


Figura 5: Caso 5: 3 estudiantes con notas 07, 11, 14. Promedio esperado: 10, Max: 14, Min: 07.

4.3. Comparación de resultados

Caso	Notas	Prm. esper.	Prm. obt.	Max/Min esper.	Max/Min obt.
1	15, 18, 20	17	17	20 / 15	20 / 15
2	10, 12, 14, 16	13	13	16 / 10	16 / 10
3	20, 20, 20, 20, 20	20	20	20 / 20	20 / 20
4	05, 00	2	2	05 / 00	05 / 00
5	07, 11, 14	10	10	14 / 07	14 / 07

Cuadro 2: Comparación entre resultados esperados y obtenidos.

Como se observa en la tabla, todos los resultados obtenidos coinciden exactamente con los que se esperan, lo que demuestra que el programa funciona de manera correcta.

Conclusiones

El programa desarrollado cumple con el objetivo de gestionar notas mediante ensamblador 8086, ya que permite ingresar hasta 10 notas, también valida que cada una esté en el rango permitido (00–20), y calcula correctamente el promedio, así mismo se observan tanto la nota más alta como la nota más baja.

La implementación con instrucciones básicas del 8086 como `MOV`, `CMP`, `ADD` y saltos condicionales demuestra que con el emulador se puede crear varios programas sencillos pero a la vez funcionales haciendo uso del lenguaje ensamblador para resolver problemas de lógica.

Se puede concluir que el programa es funcional, confiable y cumple con todos los requisitos planteados en el objetivo del proyecto.